ArrayList vs LinkedList

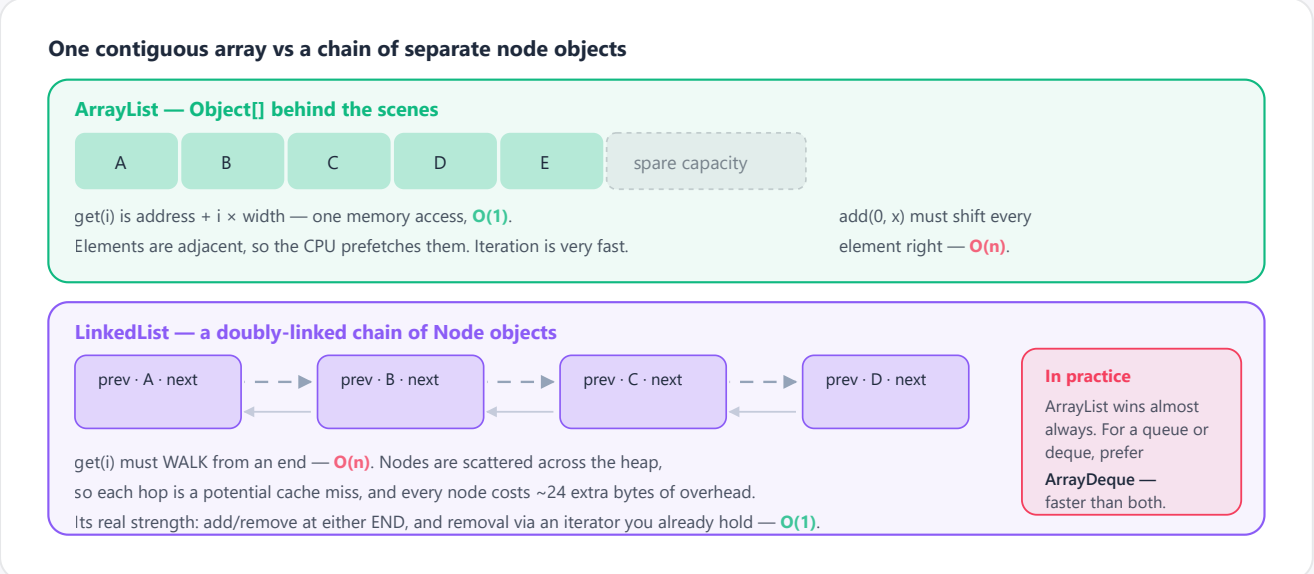
8 min read · Updated 28 Jul 2026 · Topic 17 of 65

Why it matters

Every candidate can recite the Big-O table for this comparison, and most draw the wrong practical conclusion from it. `LinkedList` has better asymptotic complexity for insertion, and is nevertheless slower than `ArrayList` for almost every real workload. Being able to explain *why* the theory misleads here — memory layout and cache behaviour — is one of the clearest ways to sound like an engineer rather than a textbook.

The concept, simply

`ArrayList` stores elements in a single `Object[]`, resized by copying when full. `LinkedList` stores each element in its own `Node` object holding `prev`, `item` and `next` pointers.



The picture explains the performance. Contiguous memory means index arithmetic and CPU prefetching; a pointer chain means a potential cache miss per element and ~24 bytes of overhead per node.

Operation	`ArrayList`	`LinkedList`
`get(i)` / `set(i)`	**O(1)**	O(n) — walks from the nearer end
`add(e)` at the end	O(1) amortised	O(1)
`add(0, e)` at the front	O(n) — shifts everything	**O(1)**
`add(i, e)` in the middle	O(n) shift	O(n) to *find*, then O(1)
`remove` via an iterator you hold	O(n) shift	**O(1)**
`contains(e)`	O(n)	O(n)
Memory per element	1 reference (+ spare capacity)	**~40 bytes** — node object + 3 references

The row that decides real cases is the last one. A `LinkedList` node is a separate heap object with its own header, so each element costs roughly 40 bytes against 4–8 for an `ArrayList` slot. Worse, those nodes are scattered wherever the allocator put them, so walking the list is a sequence of unpredictable memory jumps. An `ArrayList`'s elements sit adjacent, and modern CPUs prefetch the next cache line automatically — often making an O(n) array shift, which is a single fast `System.arraycopy`, beat an O(1) linked insertion that had to be located first.

ArrayList growth: the default capacity is 10 (allocated lazily on first add). When full, it allocates a new array of about 1.5× the size and copies. That growth is amortised O(1) per add, but each individual resize is O(n) and allocates a fresh array — which is why presizing matters when you know the size.

JAVA Copy

```java
// Presize when you know roughly how many elements are coming.
List<String> list = new ArrayList<>(10_000);   // no intermediate resizes

// The O(n²) trap: removing by value in a loop shifts on every removal.
for (int i = 0; i < list.size(); i++) {
    if (shouldRemove(list.get(i))) list.remove(i--);   // ✗ shifts + fiddl
}

// ✓ Java 8+: one pass, handled correctly by the implementation.
list.removeIf(this::shouldRemove);
```

The LinkedList trap that catches everyone — O(1) insertion is only O(1) if you are already there:

JAVA Copy

```java
LinkedList<Integer> list = new LinkedList<>(/* 100k elements */);

// ✗ O(n²). "Insertion is O(1)" is true only once you HAVE the node — and
//   get(i)/add(i, e) must walk the chain to find it first.
for (int i = 0; i < list.size(); i++) {
    list.add(i, compute(i));
}

// ✓ O(n). The iterator already holds the current node, so add/remove
//   through it really is O(1).
ListIterator<Integer> it = list.listIterator();
while (it.hasNext()) {
    int v = it.next();
    if (v < 0) it.remove();       // genuinely O(1)
}
```

And what to use instead when you thought you wanted a `LinkedList`:

JAVA Copy

```
// a growable circular array; no per-element node objects.
Deque<Task> stack = new ArrayDeque<>();
stack.push(task);                  // LIFO
stack.pop();

Deque<Task> queue = new ArrayDeque<>();
queue.offer(task);                 // FIFO
queue.poll();

// LinkedList's honest niche is narrow: you need List AND Deque in one typ
// or you are doing heavy iterator-based splicing in the middle of a big l
```

Under the hood

`ArrayList` **removal is a single** `System.arraycopy`, which is an intrinsic — the JIT replaces it with a tight, vectorised memory move rather than a Java loop. Shifting 10,000 references is genuinely fast in absolute terms, which is the mechanical reason the Big-O comparison misleads. Complexity classes describe growth, not constants, and here the constants differ by an order of magnitude in `ArrayList`'s favour.

Both are fail-fast and neither is thread-safe. Concurrent structural modification throws `ConcurrentModificationException` on the next iteration step (last topic in this chapter). For concurrent use, `CopyOnWriteArrayList` suits read-heavy workloads with rare writes; otherwise use explicit locking or a concurrent queue.

`subList` **is a view, not a copy**, which surprises people and is occasionally exploited deliberately. It writes through to the backing list, and structurally modifying the parent invalidates it. `list.subList(0, 5).clear()` is a legitimate idiom for removing a range in one call — and an accidental way to destroy data if you thought you had a copy.

Common mistakes

- **Choosing** `LinkedList` **because "insertion is O(1)".** It is O(1) only at the ends or through an iterator you already hold. Positional insertion still walks the chain.
- **Calling** `get(i)` **on a** `LinkedList` **in a loop.** That is O(n^2). Iterate with the for-each loop or an iterator.

- **Removing inside an index loop.** Skips elements and shifts repeatedly. Use `removeIf` or an iterator's `remove()`.
- **Not presizing a large `ArrayList`.** Repeated 1.5× growth means several array allocations and copies.
- **Using `LinkedList` as a queue.** `ArrayDeque` is faster and uses far less memory.
- **Treating `subList` as a copy.** It is a live view.
- **Ignoring the memory difference.** A million-element `LinkedList` costs tens of megabytes more than the equivalent `ArrayList`.

Interview lens

Q: "`ArrayList` vs `LinkedList` — when would you use each?"
A: `ArrayList` for essentially everything. It is backed by a contiguous array, so indexed access is O(1) and iteration is cache-friendly. `LinkedList` only wins for O(1) insertion and removal at the ends or through an iterator you already hold — and even then, if I want a queue or stack I would use `ArrayDeque`, which beats it on both counts. In practice I reach for `LinkedList` almost never.

Q: "But `LinkedList` has O(1) insertion — why is it slower?"
A: Two reasons. First, that O(1) applies only once you are positioned at the node; `add(i, e)` must walk there, so positional insertion is O(n) anyway. Second, constants dominate at realistic sizes: `ArrayList`'s shift is one `System.arraycopy`, a JIT intrinsic on contiguous memory that the CPU prefetches, while `LinkedList` chases pointers to scattered objects and misses cache on each hop, with about 40 bytes of overhead per element.

Q: "How does `ArrayList` grow?"
A: It starts at capacity 10, allocated lazily on the first add. When full it allocates a new array of roughly 1.5× the size, copies everything over with `System.arraycopy`,

and drops the old one. That makes `add` amortised O(1), though any individual add can be O(n). Passing an initial capacity avoids the intermediate allocations entirely.

Q: "How do you remove elements while iterating?"

A: `removeIf(predicate)` is the clearest. Otherwise use an explicit `Iterator` and call `it.remove()`, which updates the iterator's expected modification count. Never call `list.remove( ... )` inside a for-each loop — that throws `ConcurrentModificationException`. And never use an index loop with `remove(i)`, which silently skips elements as everything shifts left.

Q: "Is `ArrayList` thread-safe? What are the alternatives?"

A: No. `Collections.synchronizedList` wraps it with a lock per method, though you still need external synchronisation around iteration. `CopyOnWriteArrayList` is better when reads vastly outnumber writes — every write copies the whole array, so iteration is lock-free over an immutable snapshot and never throws. For a producer/consumer workload I would use a concurrent queue instead of a list.

Q: "What does `subList` return?"

A: A view onto the original list, not a copy — writes go through to the backing list, and structurally modifying the parent invalidates the view. That makes `list.subList(a, b).clear()` a neat way to delete a range, and makes it a trap if you assumed you had an independent copy. Wrap it in `new ArrayList<>( ... )` when you want one.

Cheat-card

- **Default to `ArrayList`.** Contiguous array, O(1) index, cache-friendly.
- `LinkedList`: O(1) only at the **ends** or via a held **iterator**. `get(i)` is O(n).
- ~40 bytes/element for LinkedList vs 4–8 — plus cache misses on every hop.
- Growth: capacity 10 → **×1.5** with `System.arraycopy`. Presize when you can.

- `subList` is a **view**. Both classes are fail-fast and not thread-safe.

QUICK CHECK — no marks, just for you.

1. LinkedList has O(1) insertion. Why is it still usually slower than ArrayList in practice?

- ◯ Because LinkedList does not implement List properly
- ◯ Because its O(1) applies only at the ends or through an iterator you already hold — and because pointer-chasing to scattered nodes misses cache while ArrayList's shift is one contiguous System.arraycopy
- ◯ Because LinkedList is synchronised
- ◯ Because ArrayList caches its elements in the CPU register file

Check answer

2. How does ArrayList grow when it runs out of capacity?

- ◯ It converts to a LinkedList
- ◯ It doubles the array on every add
- ◯ It allocates a new array of roughly 1.5 times the size and copies with System.arraycopy — amortised O(1) per add
- ◯ It allocates one extra slot each time

Check answer

3. What does list.subList(0, 5) return?

A new ArrayList with the same elements

An independent copy of the first five elements

An immutable snapshot

A live VIEW onto the original list — writes go through, and structurally modifying the parent invalidates it

Check answer

Previous
The Collection Hierarchy

Next
HashMap Internals

LEARN MORE

ARTICLES & DOCS

- ArrayList — API docs
  docs.oracle.com
- LinkedList — API docs
  docs.oracle.com
- ArrayDeque — API docs
  docs.oracle.com

VIDEOS



Choosing between ArrayList and LinkedList - JEP Cafe #20
Java



Java Interview Questions - ArrayList Vs LinkedList
Shristi Tech Academy

Hand-picked references for this lesson.



Java Interview Kit